

CO3 – AT3 - Assignment

J KARTHIK

192472136

CSA0911

JAVA

1. Printer Queue – Inter – Thread Communication

Program:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Printer p = new Printer();

        Thread user = new Thread() -> {
            for (int i = 1; i <= 5; i++)
                try { p.submit("Document " + i); }
                catch (Exception e) {}
        };

        Thread printer = new Thread() -> {
            for (int i = 1; i <= 5; i++)
                try { p.print(); }
                catch (Exception e) {}
        };

        user.start();
        printer.start();
    }
}

class Printer {
    Queue<String> q = new LinkedList<>();
    int size = 3;

    synchronized void submit(String doc) throws InterruptedException {
        while (q.size() == size) wait();
        q.add(doc);
    }
}
```

```
        System.out.println("Submitted: " + doc);
        notify();
    }

    synchronized void print() throws InterruptedException {
        while (q.isEmpty()) wait();
        System.out.println("Printing: " + q.remove());
        notify();
    }
}
```

Output:

Submitted: Document 1

Printing: Document 1

Submitted: Document 2

Printing: Document 2

Submitted: Document 3

Printing: Document 3

Submitted: Document 4

Submitted: Document 5

Printing: Document 4

Printing: Document 5

2.Hospital Resource Allocation – Synchronization and Communication

Program:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Printer p = new Printer();

        Thread user = new Thread() -> {
            try {
                p.submit("Document-1");
                p.submit("Document-2");
            } catch (Exception e) {}
        });

        Thread printer = new Thread() -> {
            try {
                p.print();
                p.print();
            } catch (Exception e) {}
        });

        printer.start();
        user.start();
    }
}

class Printer {

    Queue<String> q = new LinkedList<>();

    synchronized void submit(String doc) throws Exception {
        q.add(doc);
        System.out.println("Submitted: " + doc);
        notify();
    }
}
```

```
}  
  
synchronized void print() throws Exception {  
    while (q.isEmpty())  
        wait();  
  
    String doc = q.remove();  
    System.out.println("Printing: " + doc);  
    notify();  
}  
}
```

Output:

Submitted: Document-1

Submitted: Document-2

Printing: Document-1

Printing: Document-2

3. Bank Withdrawal – Insufficient Exception

Program:

```
public class Main {  
    public static void main(String[] args) {  
        double balance = 50000, withdraw = 30000;  
  
        try {  
            if (withdraw > balance)  
                throw new InsufficientBalanceException("Insufficient Balance!");  
  
            balance -= withdraw;  
  
            System.out.println("Withdrawal Successful");  
            System.out.println("Balance = " + balance);  
        } catch (InsufficientBalanceException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}  
  
class InsufficientBalanceException extends Exception {  
    InsufficientBalanceException(String msg) {  
        super(msg);  
    }  
}
```

Output:

Withdrawal Successful

Balance = 20000.0

4. Multithreaded Number Processing

Program:

```
class Main {  
    public static void main(String[] args) {  
        int[] a = {10, 20, 30, 40, 50};  
        new Thread(() -> {  
            int s = 0;  
            for (int n : a) s += n;  
            System.out.println("Sum = " + s);  
        }).start();  
        new Thread(() -> {  
            int s = 0;  
            for (int n : a) s += n;  
            System.out.println("Average = " + (double)s / a.length);  
        }).start();  
        new Thread(() -> {  
            int m = a[0];  
            for (int n : a) if (n > m) m = n;  
            System.out.println("Maximum = " + m);  
        }).start();  
    }  
}
```

Output:

Maximum = 50

Sum = 150

Average = 30